

AUDIT REPORT

AhorraGo — Comparador de precios de supermercados chilenos

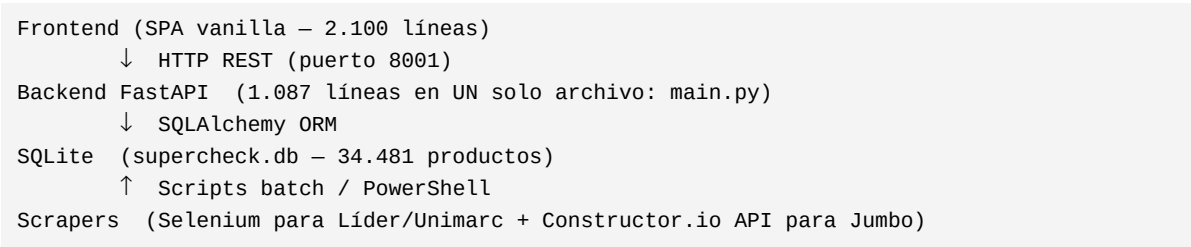
Fecha: 31 de May de 2026 | Rama: main | Último commit: 079b749

Resumen General

Métrica	Valor
Líneas de código totales	~4.600
Archivos analizados	12 principales
Vulnerabilidades críticas	2
Issues de seguridad	4
Cobertura de tests	0%
Nivel de deuda técnica	Alta

Arquitectura

El proyecto sigue una arquitectura de 3 capas con un pipeline de datos separado:



Modelo de datos



Flujo de una búsqueda

- Usuario escribe en el buscador → GET /productos/buscar/{texto}
- main.py tokeniza el texto, filtra stopwords
- Busca en BD con ILIKE por cada token
- Carga equivalentes por campo producto_base
- Recalcula clave_comparable() para agrupar variantes
- Filtra con candidato_compatible() (heurísticas ~80 líneas)
- Devuelve JSON con precios por supermercado

Problema clave: 'Leche Soprole 1L' vs 'Leche Soprole 1000ml' = NO MATCH porque la normalización de formatos es frágil.

■ Stack Tecnológico

Capa	Tecnología	Versión
Framework API	FastAPI	0.110+
ORM	SQLAlchemy	2.0+
Base de datos	SQLite	3.x
Validación	Pydantic	2.0+
Scraping web	Selenium	4.18+
Scraping API	Constructor.io	REST
Frontend	HTML + CSS + JS vanilla	—
UI Kit	Bootstrap 5	CDN
Servidor dev	Uvicorn	0.27+
Automatización	batch + PowerShell	Windows

■ Crítico

1. API Key de Jumbo hardcodeda en el código

En `app/scrapers/scrapper_jumbo_real.py`:

```
API_KEY = "key_JopvNXKS61kwGkBe" # expuesta en git history
```

La clave queda en el historial de git de forma permanente. Si el repositorio se hace público o la clave se rota, el scraper de Jumbo deja de funcionar. **Solución:** mover a variable de entorno en un archivo `.env` (excluido de git).

2. Cobertura de tests: 0%

No existe ningún archivo en el directorio `tests/`. No hay cobertura de la lógica crítica: normalización de texto, matching de productos ni cálculo de ahorro óptimo. Cualquier cambio en el core puede introducir regresiones sin detección.

■ Alto

3. main.py monolítico — 1.087 líneas

Toda la lógica de negocio vive en un solo archivo: endpoints, normalización de texto, matching heurístico, cálculo de ahorro, y más de 400 líneas de reglas hardcodedas. Es difícil de mantener, testear y extender.

4. Normalización duplicada

`normalizar_texto()`, `clave_comparable()` y `generar_producto_base()` están duplicadas entre `main.py` e `importar_csv.py`. Si se cambia la lógica en uno, el otro queda desincronizado generando datos inconsistentes.

5. Sin paginación en búsqueda

`db.query(Producto).filter(...).all()` sin LIMIT. Con 34K productos, una búsqueda genérica puede retornar miles de resultados en una sola respuesta, saturando memoria y tiempo de respuesta.

6. N+1 queries y recálculo en loop

Para cada candidato encontrado se recalcula `clave_comparable()` iterativamente. Los equivalentes se cargan todos en memoria sin filtros adicionales en BD.

7. Matching de productos frágil

`candidato_compatible()` tiene ~80 líneas de reglas manuales con más de 100 marcas hardcodeadas. Ejemplo de fallo conocido: 'Leche Soprole 1L' vs 'Leche Soprole 1000ml' = NO MATCH por diferencia de formato.

■ Medio

8. CORS demasiado permisivo

El regex de CORS permite cualquier IP en la red local (192.168.x.x / 10.x.x.x). En producción debe restringirse a orígenes explícitos.

9. Sin rate limiting

La API es pública sin throttling. Un cliente puede realizar miles de requests sin restricción, exponiendo la BD a scraping y abuso.

10. `check_same_thread=False` en SQLite

Desactiva la protección de threading de SQLite. Bajo carga concurrente con Uvicorn async, existe riesgo de corrupción de datos.

11. Frontend en un solo archivo (2.100 líneas)

`index.html` mezcla HTML, 900+ líneas de CSS inline y 1.200+ líneas de JS. Sin separación de concerns, sin minificación, sin caché de assets estáticos.

12. Lock file de actualización no funcional

`actualizacion_productos.lock` se crea pero no bloquea nada. Dos ejecuciones simultáneas del pipeline pueden corromper la base de datos.

13. Logs sin rotación

Cada ejecución del pipeline de actualización crea un archivo de log nuevo sin límite ni rotación, lo que puede llenar el disco con el tiempo.

14. Scrapers sin rate limiting

Sin delays entre requests y sin validación de robots.txt. Riesgo de ban de IP y posible violación de los términos de servicio de Líder y Unimarc.

15. URLs sin escape correcto

services.py usa `query.replace(' ', '%20')` que no escapa tildes ni caracteres especiales como ñ, á, é — generando URLs malformadas en búsquedas en español.

■ Deuda Técnica

Área	Problema	Impacto
Índices BD	Faltan en <code>precio.supermercado_id</code> y <code>producto.producto_base</code>	Performance
Normalización	Lógica duplicada <code>main.py</code> ↔ <code>importar_csv.py</code>	Consistencia de datos
Frontend	2.100 líneas en un solo archivo	Mantenibilidad
Matching	Heurísticas hardcodeadas (100+ marcas)	Escalabilidad
Backups	CSV y BD no sincronizan en fallo → estado inconsistente	Integridad
Validación de precios	Umbrales por familia hardcodeados en código	Fragilidad
URLs scraping	Escape incompleto de caracteres especiales	Bugs silenciosos
API Key	Expuesta en git history	Seguridad
Tests	Cobertura 0%	Regresiones

■ Recomendaciones Priorizadas

Inmediato — esta semana

- **Mover API key a variable de entorno:** crear `.env` con `JUMBO_API_KEY` y agregarlo a `.gitignore`
- **Agregar LIMIT a queries de búsqueda:** default 50, configurable por parámetro
- **Implementar lock file real:** usar librería `filelock` antes de correr el pipeline de actualización

Corto plazo — 2 semanas

- **Extraer `app/normalizacion.py`:** módulo compartido importado desde `main.py` e `importar_csv.py`
- **Tests unitarios mínimos:** normalización de texto, matching de productos, cálculo de ahorro
- **Agregar `slowapi` para rate limiting:** 10–20 req/min por IP en endpoints de búsqueda
- **Agregar índice faltante:** `CREATE INDEX idx_precio_supermercado ON precios(supermercado_id)`
- **Escape correcto de URLs:** usar `urllib.parse.quote()` en lugar de `replace(' ', '%20')`

Mediano plazo — 4 semanas

- **Separar frontend:** `style.css` + `app.js` + `index.html`
- **Fragmentar `main.py`:** `routers/busqueda.py`, `routers/carrito.py`, `services/matching.py`
- **Fuzzy matching con Levenshtein:** reemplazar heurísticas manuales de formatos (1L vs 1000ml)

- **Paginación estándar:** limit + offset en todos los endpoints de listado
- **Rotación de logs:** RotatingFileHandler con límite de 10MB por archivo

■ Conclusión

AhorraGo es un **MVP funcional** con el core de negocio operativo. Los dos blockers reales para producción son la **API key expuesta en el repositorio** y los **cero tests** — el resto son problemas de escalabilidad y mantenibilidad que crecen con el tiempo pero no rompen nada hoy.

El costo de deuda técnica más alto a largo plazo es el **main.py monolítico**: cada feature nueva lo hace más difícil de mantener y testear. Se recomienda fragmentarlo antes de iniciar la Fase 1 del roadmap (sistema de usuarios).

La base de datos está dimensionada para el volumen actual (34K productos). Si se escala a 200K+, se requerirán índices adicionales y eventualmente migrar a PostgreSQL.